CS 486/686
Dissecting a Small Language Model

Yuntian Deng

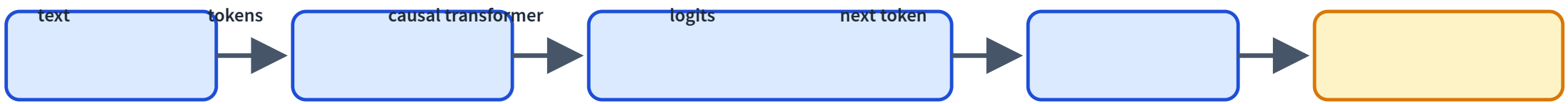
Lecture 21

Opening the hood on Qwen3-0.6B

Learning goals

- Load or inspect a **small causal LM**.
- Trace one generated token at a time.
- See how **decoding** and **prompting** change behavior.
- Recognize limitations and failure modes.

From L20 to L21



L20 taught the pipeline. Today we watch it run.

The model we will dissect

Qwen3-0.6B

a real small causal language model

0.6B parameters	**28** layers	**32k** context
~151k vocab	**GQA** 16 Q / 8 KV	**BBPE** tokenizer

A "small" LM is still a serious learned system.

What gets loaded?

tokenizer

text ↔ token IDs

config

layers, heads, vocab

weights

learned parameters

settings

temperature, max tokens

Demo path and fallback

optional live path

Transformers.js + ONNX + WebGPU

reliable path

precomputed traces embedded here

The concepts do not depend on a live GPU demo working.

Optional browser inference

```
import { pipeline, TextStreamer } from "@huggingface/transformers";

const generator = await pipeline(
  "text-generation",
  "onnx-community/Qwen3-0.6B-ONNX",
  { device: "webgpu", dtype: "q4f16" },
);
```

This loads a web-ready tokenizer and quantized model weights.

The same pipeline in Python

```
tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen3-0.6B",
    torch_dtype="auto",
    device_map="auto",
)
```

Browser and Python paths expose the same conceptual pieces.

Start with a prompt

```
Explain gradient descent in one sentence.
```

Inference begins as human-readable text.

The model sees a chat template

system	You are a helpful assistant.
user	Explain gradient descent in one sentence.
assistant	

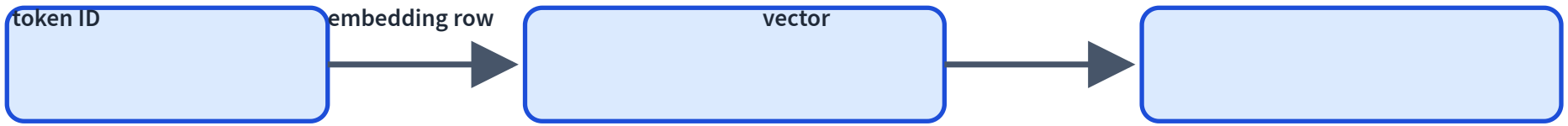
The model's actual input is usually not exactly what the user typed.

Prompt to tokens

`Explain` `gradient` `descent` `in` `one` `sentence` `.`

token	illustrative id
Explain	8493
gradient	33497
descent	41184

Tokens become embeddings



Each token begins generation as a learned vector.

Through 28 transformer blocks

Attention: communicate across context.

Feed-forward: per-token computation.

Residuals and normalization keep the deep stack stable.

The last position predicts next

For generation, the next distribution comes from the final prompt position.

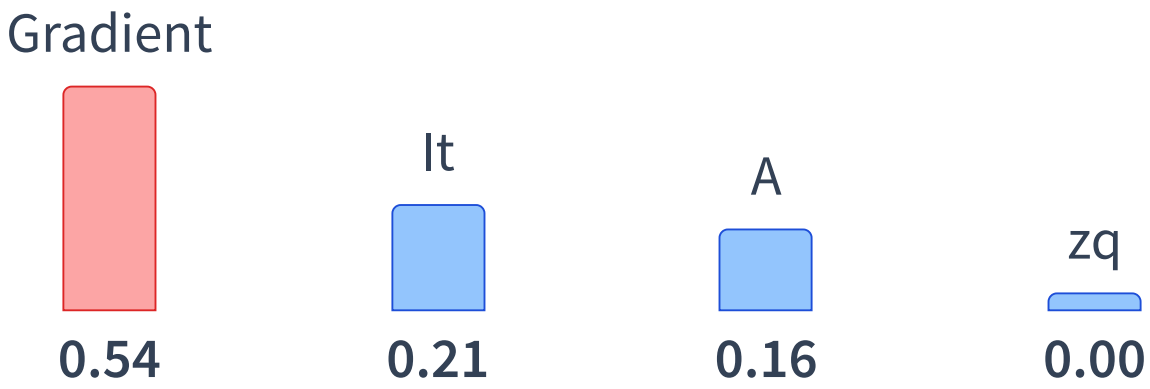
Logits: top candidates

Illustrative trace after the prompt: `Explain gradient descent in one sentence.`

candidate token	logit	interpretation
Gradient	8.4	high
It	6.8	medium
A	6.3	medium
zq	-2.5	very low

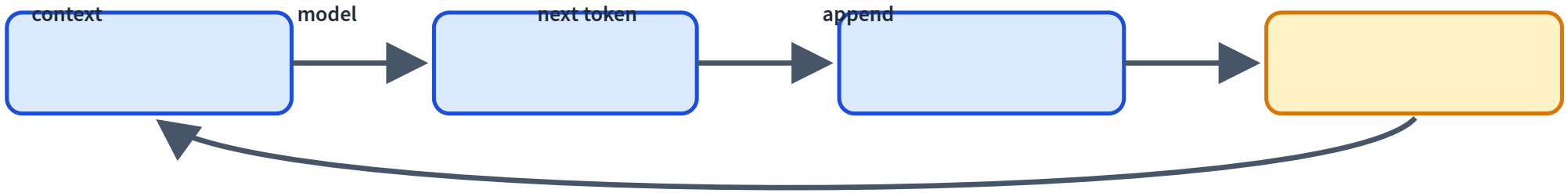
The exact values here are a teaching trace, not a live benchmark.

Softmax and decoding choice



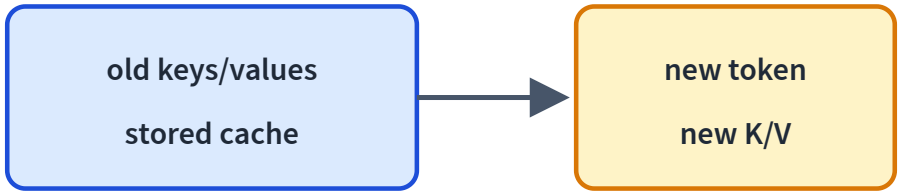
Decoding turns scores into one actual next token.

Append and repeat



Generation is autoregressive and sequential at inference time.

KV cache: do not recompute everything

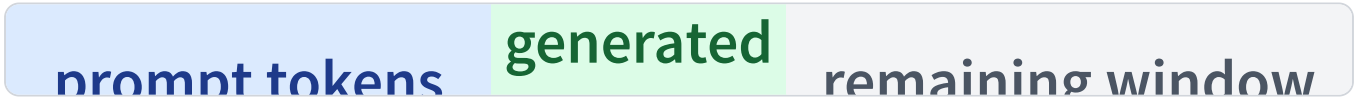


Generated tokens keep their keys/values.

The next step computes only the new token's pieces.

This is why long context costs memory.

Context length is working memory



For Qwen3-0.6B: up to about **32k tokens**.

It is not permanent memory; it is the current working context.

Decoding changes behavior

greedy

deterministic, safe, repetitive

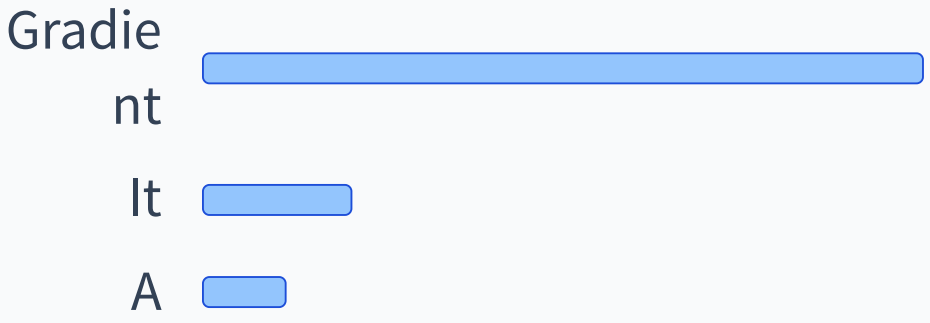
sampling

varied, sometimes better, sometimes worse

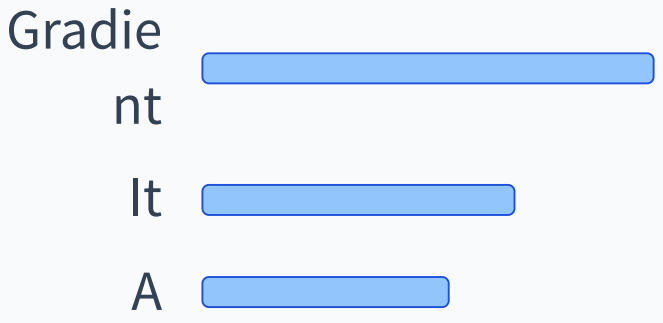
Weights stay fixed; only the token selection rule changes.

Temperature controls randomness

low T



high T



Top-k and top-p trim the tail

top-k

sample from the k most likely tokens

top-p

sample from tokens whose cumulative mass reaches p

Keep randomness inside plausible choices.

Prompting is inference-time conditioning

Prompt A

Explain gradient descent.

Prompt B

Explain gradient descent to a 10-year-old.

The context changes. The weights do not.

Few-shot prompting specifies a pattern

```
tiny → small
 huge → big
rapid → fast
```

Examples in the prompt can define a task without training.

Chain-of-thought-style prompting

direct

Answer the question.

reasoning style

Think step by step, then answer.

Reasoning traces can help behavior, but are not guaranteed faithful explanations of internal computation.

Some models expose thinking modes

non-thinking

answer directly

thinking

produce intermediate reasoning text

For Qwen-style models, special chat-template controls can change this behavior.

Failure mode: hallucination

```
Who won the 2031 Turing Award?
```

A model may answer fluently anyway.

Next-token likelihood is not truth verification.

Failure mode: prompt sensitivity

Prompt 1

Give a concise answer.

Prompt 2

Be creative and speculate.

The model is conditioned by text; small changes can matter.

Failure mode: context and cost

context

long prompts consume the window

latency

generation is token-by-token

browser

hardware support varies

What did not change today?

no fine-tuning

weights unchanged

no LoRA

no adapters

no retrieval

no external documents

We steered behavior only through inputs and decoding.

Next question

When is prompting not enough, and when do we update weights or add external knowledge?

L22: From Prompting to Fine-Tuning and LoRA.

Recap

✓ Loaded tokenizer, config, weights, and generation settings.

✓ Tokenized a prompt and traced it into embeddings.

✓ Ran a causal LM to produce logits.

✓ Decoded one token at a time.

✓ Used prompting and decoding without updating weights.

✓ Identified common failure modes.